

drifthappens.predreprod1

Experiments that motivate claims in ~/papers/Drift20205/submissions/POBAM2026 about how the effects of e.g. an internal predation process that reduces N temporarily can lead to a great deal of drift even though N stays the same from generation to generation. Proof of concept.

There are possible problems with generating the binomial probabilities---they become small, are often zero--and with how long it takes to multiply the matrices when N is greater than 1000 or 2000. For the latter problem, consider using Neanderthal

FIXME ? Because the plotly plots are including x and y axes at 0, it can be misleading that there's a line that goes up to the top of the plot at zero, when the probability of zero is less than that upper value.

TODO Possibly make the defs into lets and wrap everthing in this file in a single function. It can still be hacked at will, but that would make it easier to benchmark with critierium.

Configuration and setup

WHICH MATRIX POWERS FUNCTION? Consider using choose-mat-powers-sequentially if the exponents are closely spaced; that might be more efficient. Or consider choose-mat-powers-parallel, which is choose-matrix-powers-separately, but using pmap rather than map.

(def choose-mat-powers mats/choose-mat-powers-sequentially) (def choose-mat-powers mats/choose-mat-powers-separately)

```
(def choose-mat-powers mats/choose-mat-powers-parallel)
```

```
(def fit-B 1.0)
```

```
(def big-fit-A 1.01) ; large size has sel
```

```
(def small-fit-A 1.0) ; small size is pure drift
```

```
(def interval 16) ; interval between generations to display
```

```
(def half-interval (/ interval 2)) ; superceded by half-generations
```

```
(def increments (iterate (partial + interval) 0))
```

```
(def generations increments)
```

```
(def half-generations (map (fn [n] (/ n 2)) increments))
```

Number of generation states to display from a sequence of such states. that are generations apart. So e.g. if num-gens = 6, and generations = 16, then using the def of half-generations above, the last generation displayed is $16 * (6 - 1) = 80$ [since the first displayed generation is generation 0].

```
(def num-gens 6)
```

These next two values should be even numbers so that when divided by 2, we'll get integers:

```
(def small-N 500)
```

```
(def big-N 5000)
```

```
(def half-small-N (/ small-N 2))
```

```
(def half-big-N (/ big-N 2))
```

Small populations

```
(def small-pop-init
  "A vector of initial probabilities of frequencies for a small population,
  typically with 1 as one element and zeros elsewhere."
  (mats/mkvec (concat (repeat half-small-N 0.0) [1.0] (repeat half-small-N 0.0))))
```

```
(def small-tran-mat
  "A single transition matrix for a small population."
  (wf/right-mult-tran-mat small-fit-A fit-B (dec (count small-pop-init))))
```

```
(def small-tran-mats
  "A sequence of M-to-M transition matrices, each of which is
  (choose-mat-powers small-tran-mat (take num-gens generations))

(def small-prob-states
  "States resulting from applying a product transition matrix
  (mats/make-prob-states small-tran-mats small-pop-init))
```

Plots made from the preceding sequence of states.

```
(def small-plots (mapv uplot/plot-dots-lines
  small-prob-states
  (repeat (str "### $M=" small-N "$"))))
```

DISPLAY PLOTS: small-plots

Large populations

```
(def big-pop-init
  "A vector of initial probabilities of frequencies for a big
  typically with 1 as one element and zeros elsewhere."
  (mats/mkvec (concat (repeat half-big-N 0.0) [1.0] (repeat half-big-N 0.0))))

(def big-tran-mat
  "A single transition matrix for a big population."
  (wf/right-mult-tran-mat big-fit-A fit-B (dec (count big-pop-init))))

(def big-tran-mats
  "A sequence of N-to-N transition matrices, each of which is
  (choose-mat-powers big-tran-mat (take num-gens generations))

(def big-prob-states
  "States resulting from applying a product transition matrix
  (mats/make-prob-states big-tran-mats big-pop-init))
```

Plots made from the preceding sequence of states.

```
(def big-plots (mapv uplot/plot-lines
                    big-prob-states
                    (repeat (str "### $N=" big-N "$"))))
```

DISPLAY PLOTS: big-plots

Alternating small/large populations

```
(def N (dec (count big-pop-init)))
```

```
(def M (dec (count small-pop-init)))
```

Shrinking and expanding matrices.

Note M and N are swapped in the next two

```
(def predat-tran-mat
  "A transition matrix from a population of size N to a smaller
  of size M resulting from predation."
  (wf/right-mult-tran-mat small-fit-A fit-B N M)) ;
```

```
(def reprod-tran-mat
  "A transition matrix from a population of size M to a larger
  of size N due to reproduction."
  (wf/right-mult-tran-mat big-fit-A fit-B M N)) ; use fit-B for
```

Combine into one square matrix:

```
(def pred-reprod-mat
  "A transition matrix from a population of size N to a population
  same size. The matrix is the product of predat-tran-mat and
  reprod-tran-mat. It represents predation followed by reproductive growth."
  (fmat/mulm reprod-tran-mat predat-tran-mat))
```

We use half-generations here because each step involves two sampling processes.

So each generation is analogous to two generations in the small and big models.

```
(def pred-reprod-tran-mats
  "A sequence of N-to-N transition matrices, each of which is
  (choose-mat-powers pred-reprod-mat (take num-gens half-gens))

(def pred-reprod-prob-states
  "States resulting from applying a product transition matrix
  (mats/make-prob-states pred-reprod-tran-mats big-pop-init))
```

Plots made from the preceding sequence of states.

```
(def pred-reprod-plots (mapv uplot/plot-dots-lines
                             pred-reprod-prob-states
                             (repeat (str "### $M=" small-N "§
```

DISPLAY PLOTS: pred-reprod-plots

Plots from all three populations:

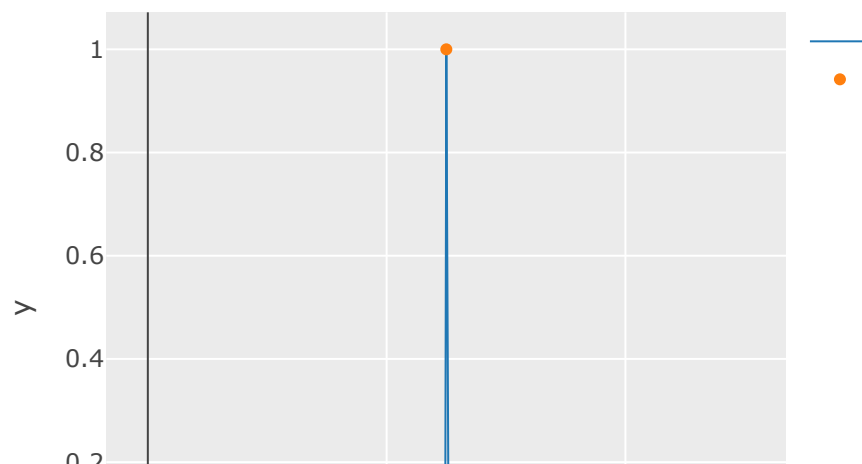
```
(def small-big-combo-plots
  "States resulting from alternating plots from other sequence
  (interleave small-plots big-plots pred-reprod-plots))
```

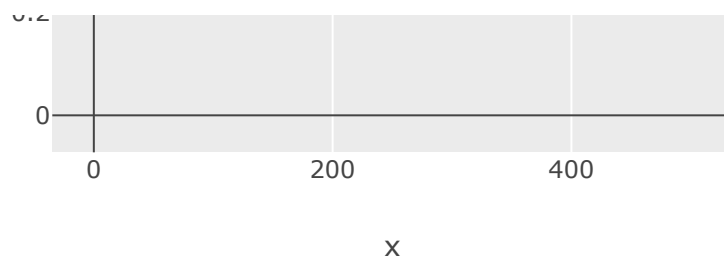
DISPLAY PLOTS:

small-big-combo-plots

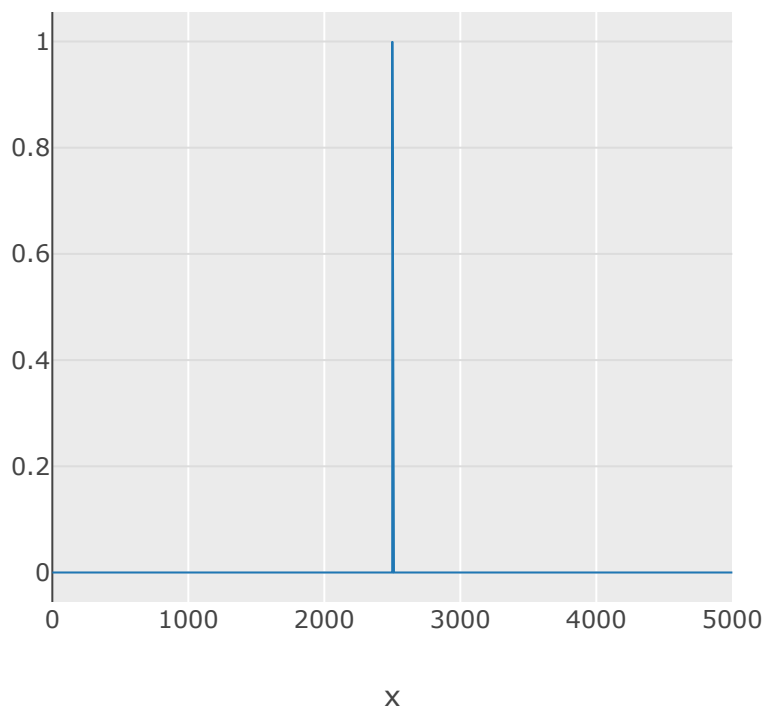
(

$$M = 500$$

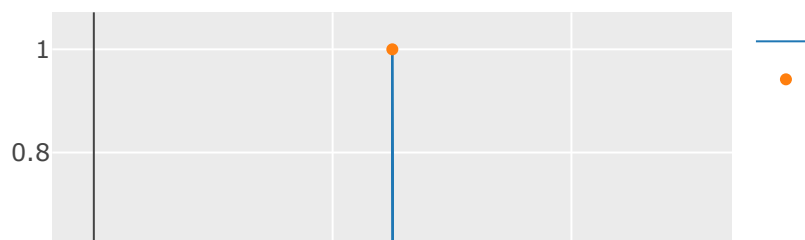


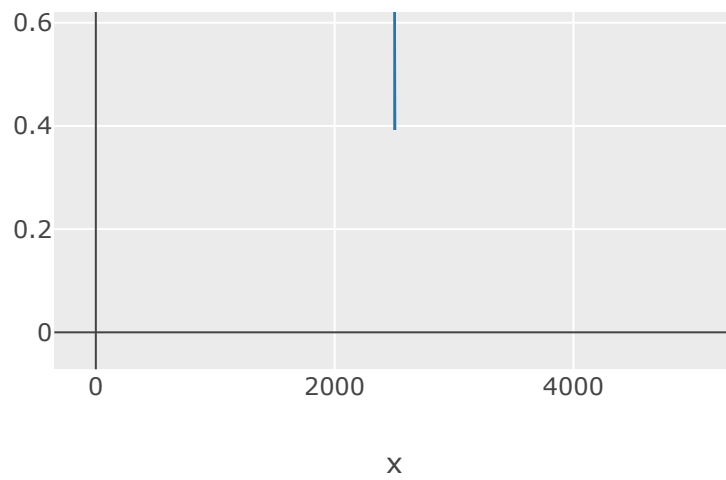


$$N = 5000$$

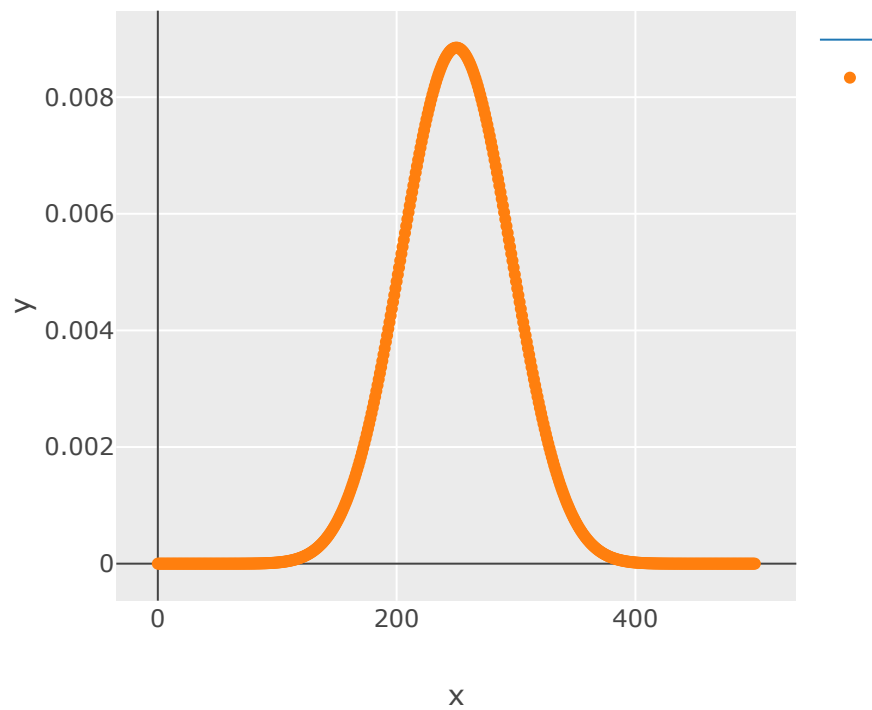


$$M = 500, N = 5000$$

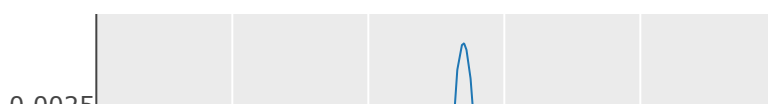


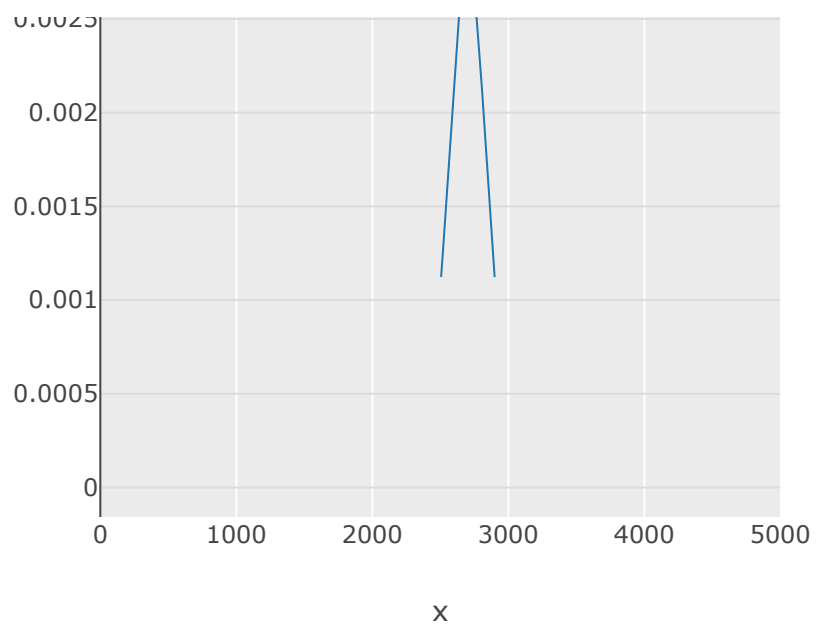


$$M = 500$$

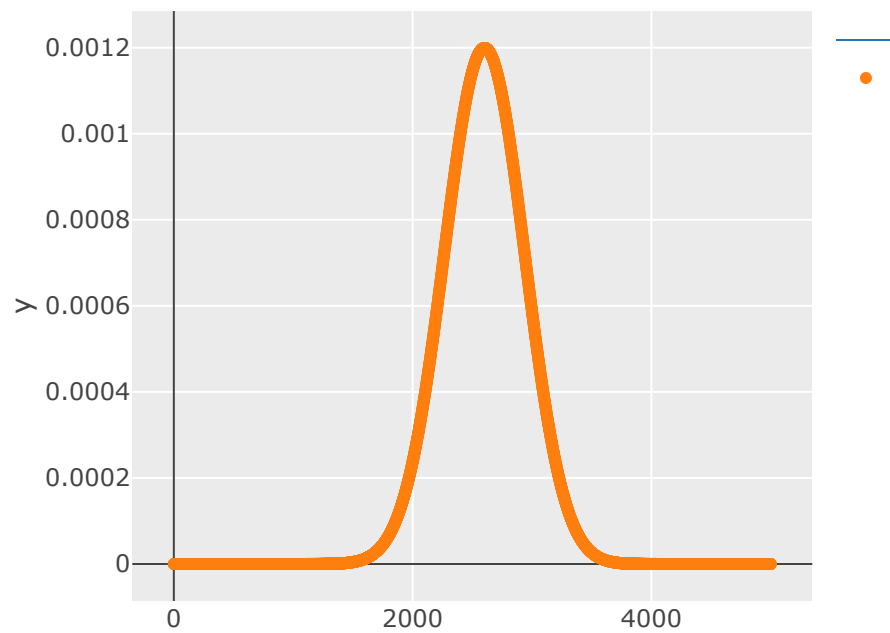


$$N = 5000$$



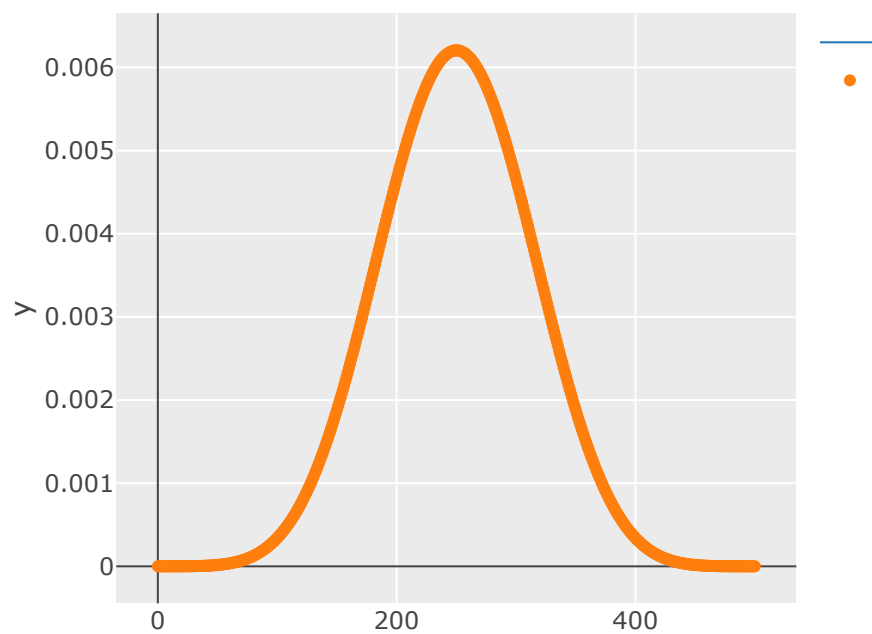


$$M = 500, N = 5000$$



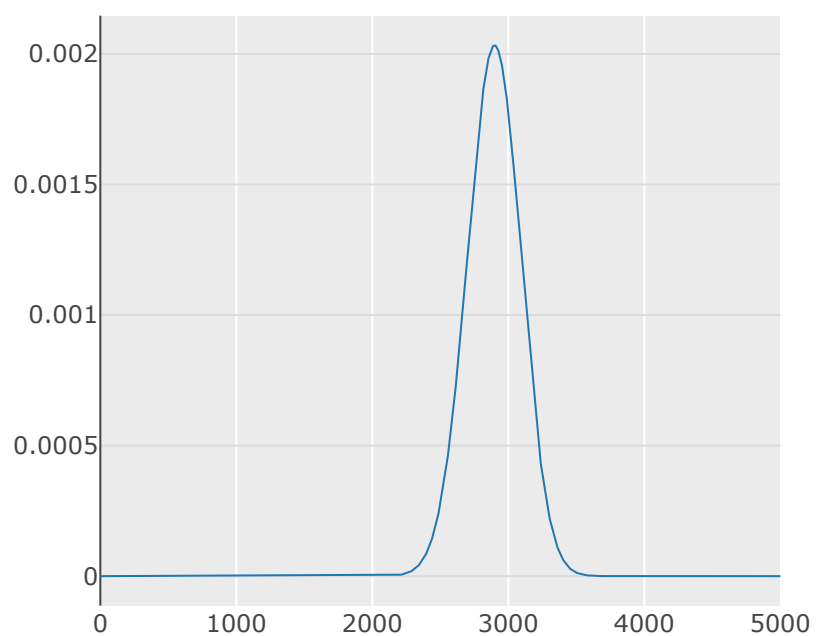
x

$$M = 500$$



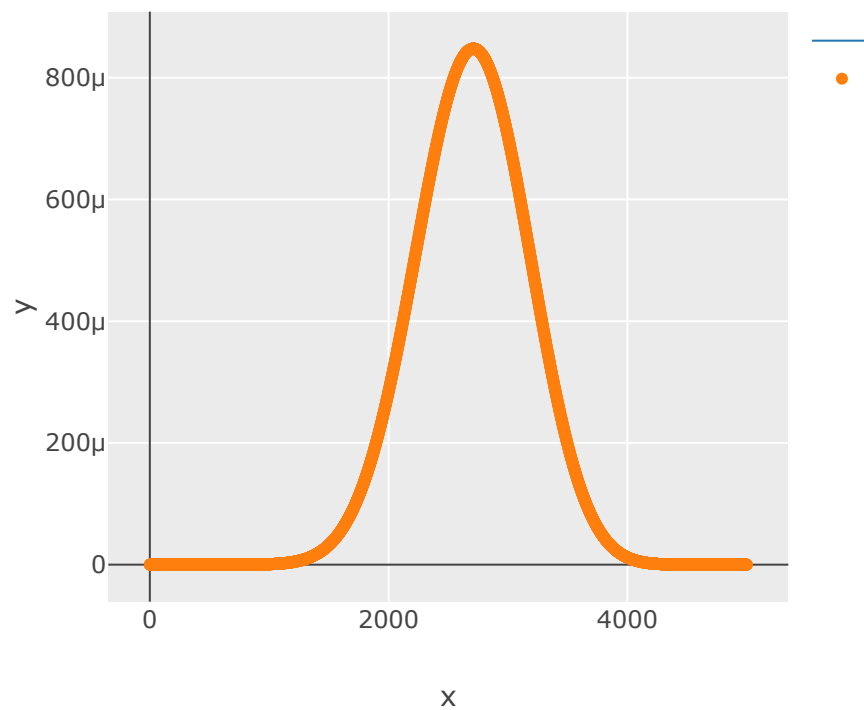
x

$$N = 5000$$

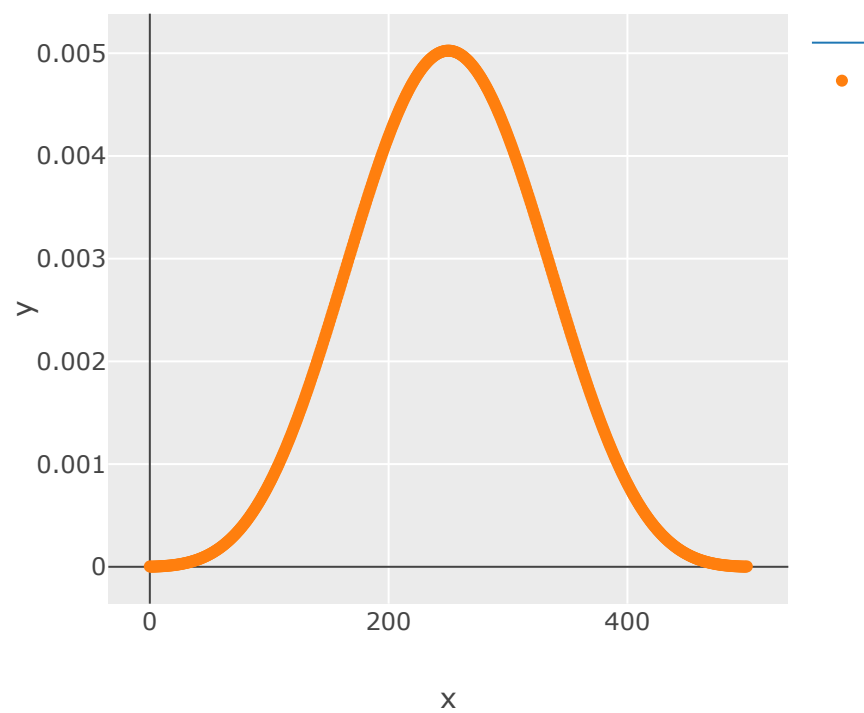


x

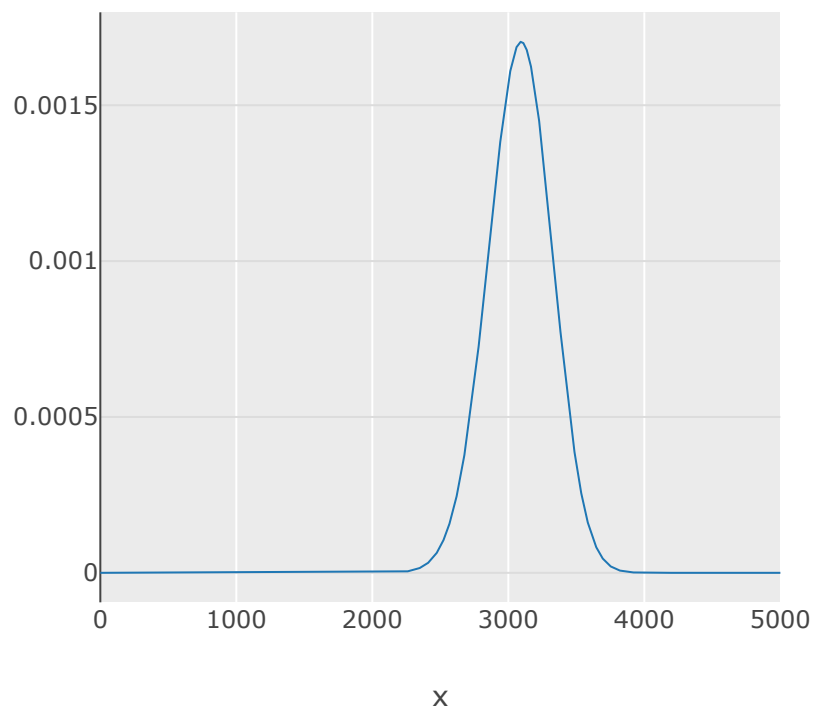
$$M = 500, N = 5000$$



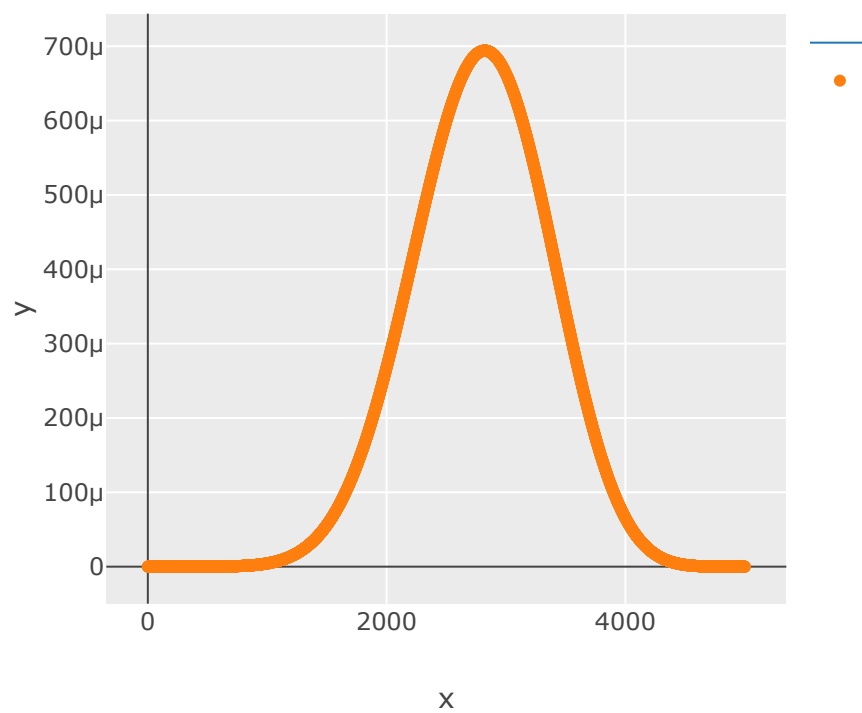
$$M = 500$$



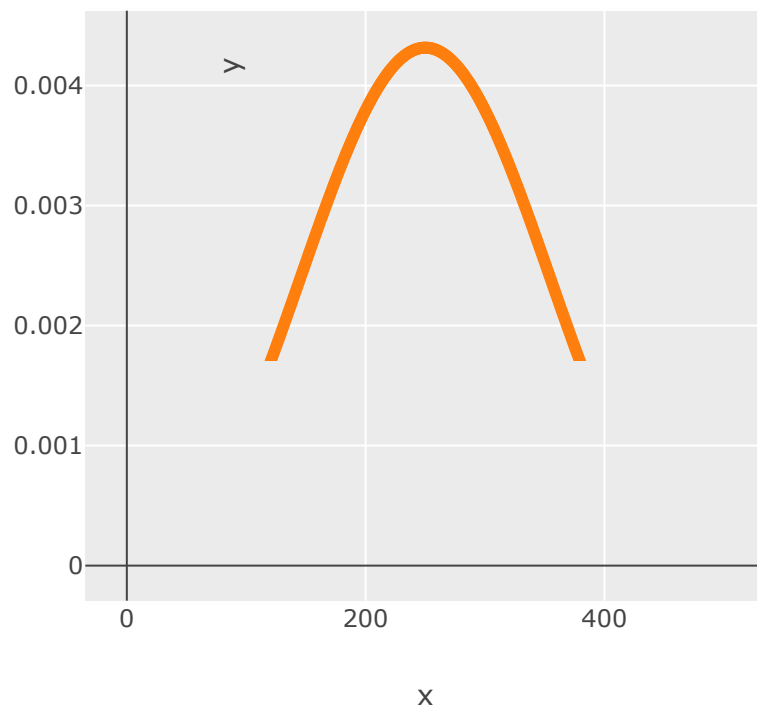
$$N = 5000$$



$$M = 500, N = 5000$$

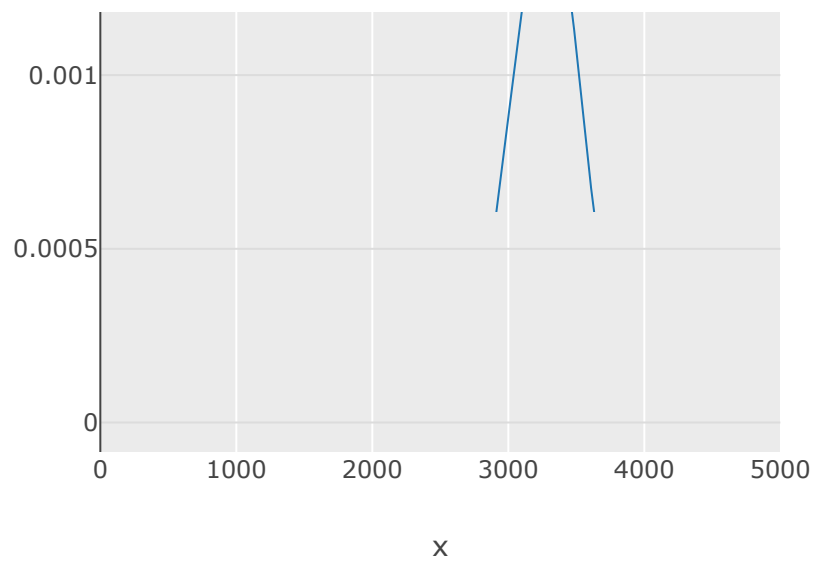


$$M = 500$$

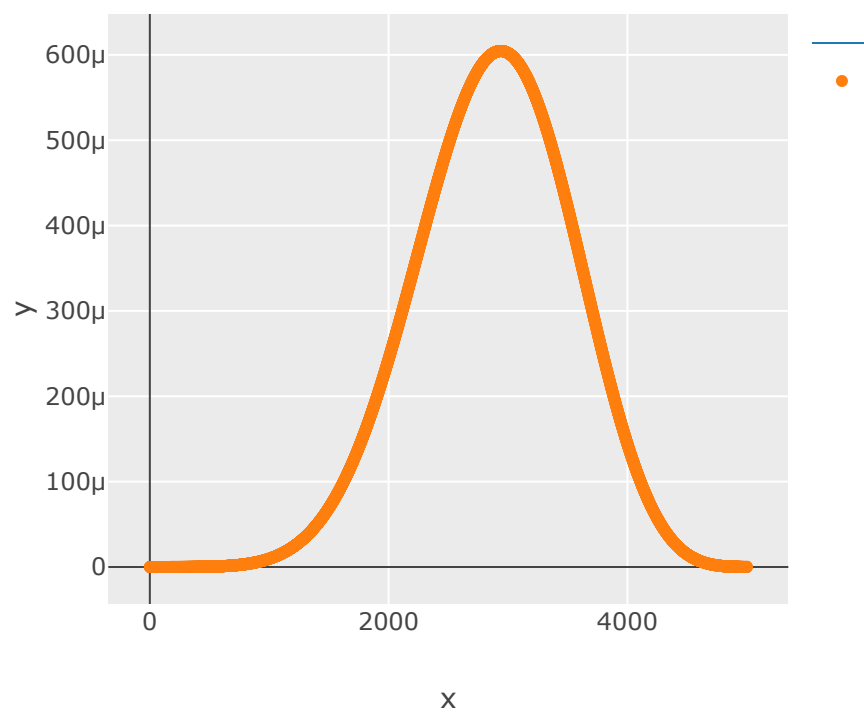


$$N = 5000$$



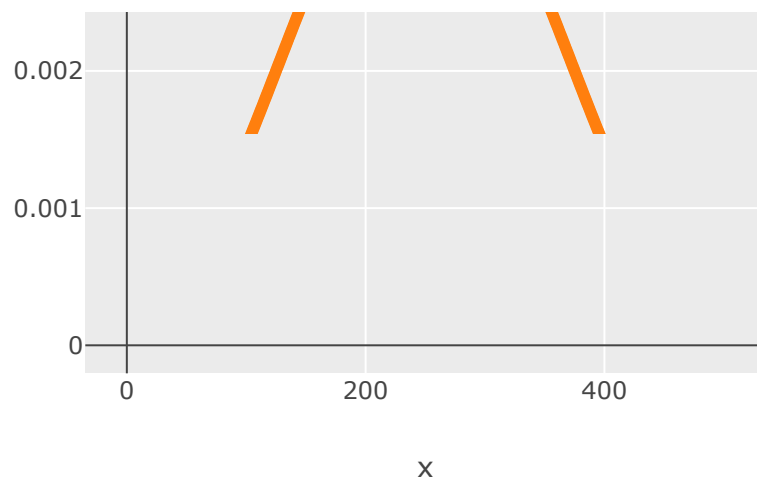


$$M = 500, N = 5000$$

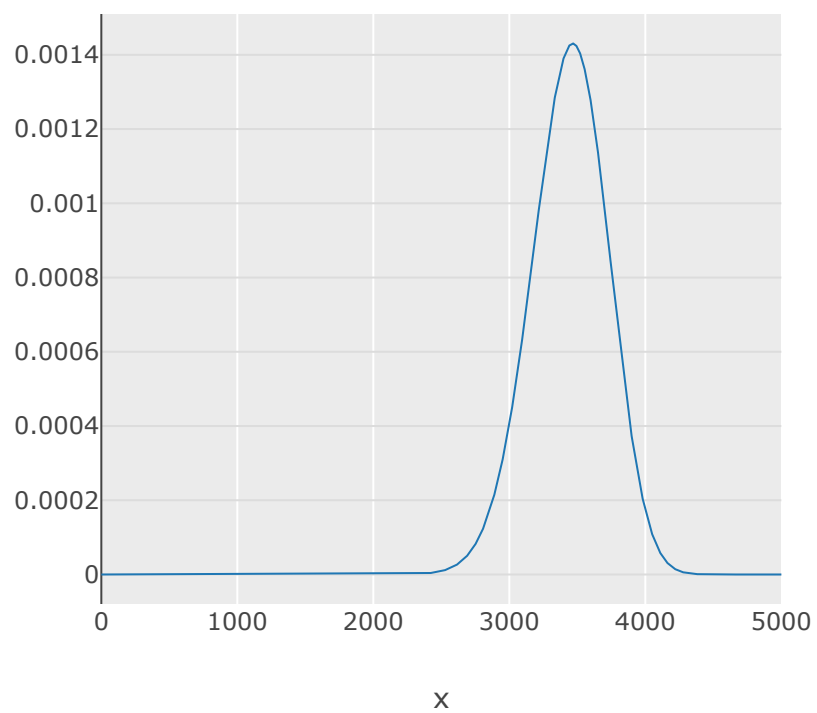


$$M = 500$$

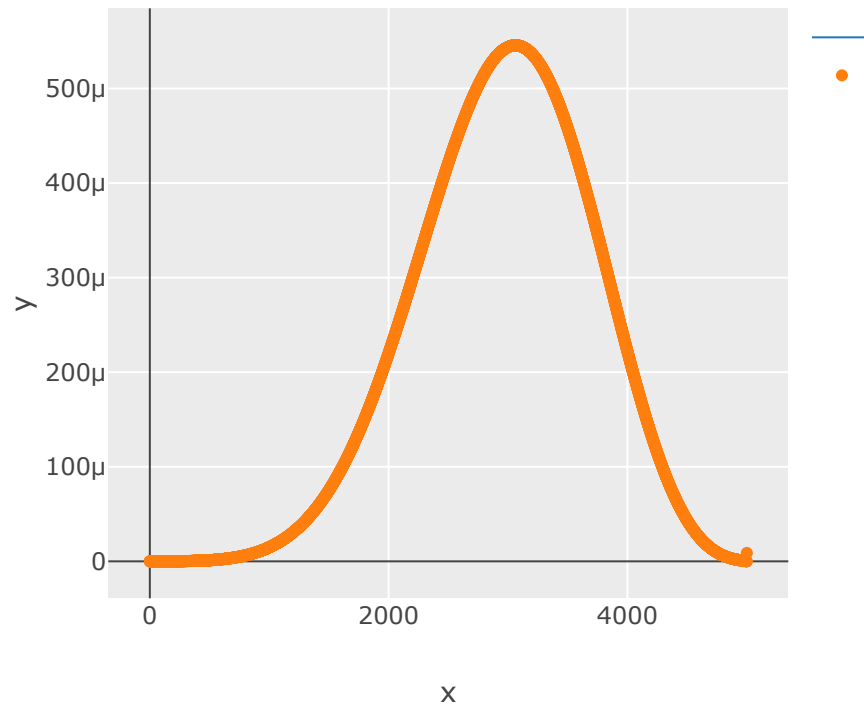




$N = 5000$



$$M = 500, N = 5000$$



)

simplistic time reporting

```
(str (java.time.LocalDateTime.now))
```

```
"2026-04-08T09:55:56.751670"
```

source: src/clj/drifthappens/predreprod1.clj